

Rapport projet PC3R



ISSue .

Track the ISS, test your geography

David VADIMON

Mai 2026

Table des matières

Table des matières	2
Introduction	3
Sujet de l'application	3
Principales fonctionnalités	3
Utilisation des APIs externes	4
Cas d'utilisations	5
Schéma de la base de données (MongoDB)	6
Description du serveur (Go)	9
authentification	9
external_apis	9
resources	9
Pour ISSHandler.go	9
Pour QuizHandlers.go	10
Pour UsersHandlers.go	10
Pour UserResponsesHandlers.go	10
Description des requêtes	11
Description du client (React)	17
/ (Home)	17
/quizzes (Quizzes)	18
/quiz/:id (QuizPage)	18
/leaderboard (LeaderboardPage)	18
/profile/:id (ProfilePage)	18
/login (Login)	19
/signup (Signup)	19
Schéma général de l'application	20
Utilisation d'IA	21

Introduction

Lien du répertoire **GitHub** : <https://github.com/00David/ISSue>

Lien du site, déployé avec **Render** : <https://issue-odz6.onrender.com/>

Sujet de l'application

Mon projet porte sur la création d'une application nommée **ISSue**, créant de manière dynamique et journalière un quiz inspiré de la position exacte de la station spatiale internationale, au moment de la génération.

Ces quiz correspondent à des QCM de 10 questions, 4 réponses proposées pour chaque question, une seule bonne réponse par question, et portent sur des faits de culture générale correspondant au lieu du quiz : le pays, la région, et de manière générale la zone géographique sont utilisés pour générer le quiz et ses questions.

Principales fonctionnalités

Voici une liste exhaustive des principales fonctionnalités du site :

- **Suivre en direct la position courante de la station spatiale internationale**, depuis la page d'accueil.
- **Répondre aux questions d'un quiz donné**. À l'issue du quiz, l'utilisateur voit alors ses résultats, et a la possibilité de **poster ses réponses optionnellement accompagnées d'un commentaire et/ou d'une note** donnée au quiz.
- **Voir ses réponses à un quiz précédemment fait**.
- **Voir les commentaires d'un quiz donné**.
- **Épingler/désépingler un quiz** en étant connecté.
- **Accéder à l'ensemble des quiz du site de 2 manières** :
 - Soit par la **liste complète des quiz**. L'utilisateur a également la possibilité de **filtrer ces quiz affichés**, et d'utiliser un **ordre d'affichage sur une des colonnes**.
 - Soit par la **carte du monde** : chaque quiz est alors placé sur la carte selon la position de l'ISS au moment du quiz.
- **Accéder au classement général du site** : les utilisateurs y sont alors tous affichés, du plus au moins performant (en nombre total de points).
- **Se créer un profil**, en fournissant son nom d'utilisateur, mot de passe, et adresse email.
- **Se connecter à son profil existant**, en fournissant son nom d'utilisateur et mot de passe.
- **Accéder au profil d'un autre utilisateur**, pour voir ses statistiques et son activité.
- **Accéder à son profil privé**, pour voir ses statistiques, son activité, **modifier ses informations personnelles**, se déconnecter ou **supprimer son profil**.

Utilisation des APIs externes

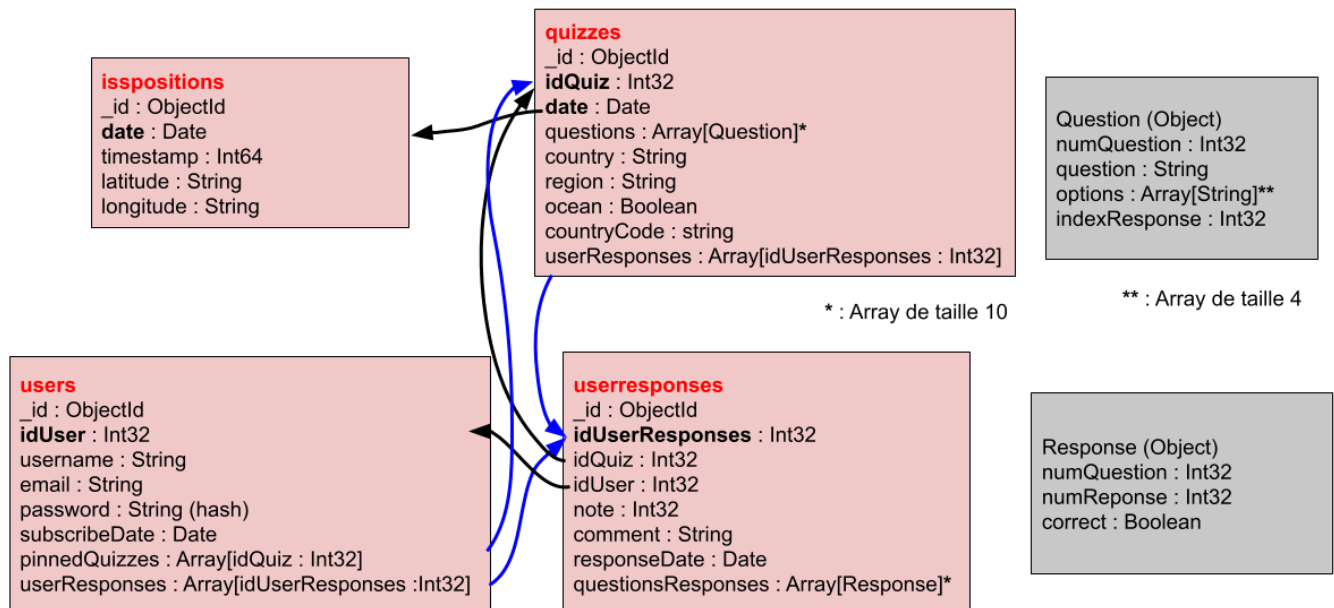
J'ai utilisé **3 API externes**, appelées toutes depuis mon serveur, dans le cadre de ce projet :

- **Open Notify** : <http://open-notify.org/Open-Notify-API/ISS-Location-Now/>
Elle permet notamment de récupérer en direct et de manière minimaliste les coordonnées de la position courante de l'ISS : lorsque l'utilisateur consulte la position courante de l'ISS sur la page d'accueil, la requête au serveur déclenche alors à son tour un appel à l'API d'Open Notify. Lors de la génération d'un quiz, un appel est également fait au début par le serveur afin de récupérer les coordonnées de l'ISS à l'instant "t", la suite étant traitée par l'API externe suivante.
Le côté dynamique est donc dans un premier temps assuré par cela.
- **BigDataCloud** : <https://www.bigdatacloud.com/free-api/free-reverse-geocode-to-city-api>
L'API offerte par ce site est assez large, mais permet essentiellement de géolocaliser ou d'effectuer à l'inverse du reverse geocoding (de l'identification géographique à partir de coordonnées données). Dans mon cas je n'utilise cette API que pour le 2ème cas : cela n'intervient que lors de la génération d'un quiz, après avoir récupéré les coordonnées de l'ISS via Open Notify, le serveur fait alors un appel à BigDataCloud en passant en paramètres de l'URL ces dernières coordonnées, et récupère une multitude d'informations sur le lieu géographique correspondant. Dans mon cas je n'en garde qu'une sous partie (notamment le pays / la région / si la zone est un océan ou non).
- **Gemini** : <https://ai.google.dev/gemini-api/docs?hl=fr>
Permettant de générer par IA à peu près tout ce qui est imaginable, j'utilise son API en tant que 3ème étape dans ma génération de quiz : après avoir eu les infos sur la zone géographique via BigDataCloud, j'effectue alors un prompt (assez gros) pour lui spécifier l'ensemble des instructions nécessaires à la création de mon quiz, comme le genre et les modalités des questions, le format du quiz, ou encore le format de retour que j'attends exactement de lui. Dans ce cas précis, je lui demande de me renvoyer un JSON contenant donc principalement le quiz et les questions, avant de le récupérer et d'**insérer le quiz et les coordonnées de la position correspondante de l'ISS en base de données**.
Le côté dynamique est donc également dans un deuxième temps assuré par cette utilisation de l'API d'une IA générative.

Cas d'utilisations

- Bob arrive sur la page d'accueil et voit que le quiz du jour concerne son pays de cœur : le Guatemala. Il décide alors de faire le quiz, obtient l'incroyable score de 10/10, décide de poster ses réponses mais se voit être redirigé sur l'écran de login afin de se connecter. Celui-ci ne possédant pas de compte, il décide alors de s'en créer un, puis de modifier son nom d'utilisateur nouvellement créé depuis son profil, revient sur l'écran d'accueil, et poste donc ses réponses.
- Benjamin se rend sur le site, accède au classement et voit alors que Bob vient de le dépasser. Il s'empresse alors de se connecter à son compte, accède à un quiz qu'il avait épinglé sur son profil, le remplit, obtient un super score, laisse une note de 5/5 au quiz avec un commentaire "TOP1 CEST MOI". Il finit par désépingler ce quiz venant d'être complété.
- Mathilde se connecte, accède à la page de l'ensemble des quiz, dans un premier temps en sélectionnant la vue de la carte du monde. Celle-ci cherchant un quiz sur l'Afghanistan, mais n'arrivant pas à le trouver sur la carte, se résout finalement à changer la vue en passant sur celle de la liste des quiz et entre dans la barre de recherche "Afghanistan", affichant alors un quiz datant du 16 mai 2026. Elle accède à ce quiz, le remplit, obtient le score de 0/10, poste ses réponses et va sur sa page de profil. En voyant ses statistiques, notamment son score moyen de 2/10 pour 30 quiz répondus, elle entre alors dans une importante phase d'introspection, la menant à se décider à supprimer son profil : ainsi l'ensemble de ses réponses sont également supprimées du site.

Schéma de la base de données (MongoDB)



> Un document de la '**collectionA**' possède un 'champA' qui référence un autre document de la '**collectionB**' avec un 'champA' identique

> Un document de la '**collectionA**' possède une 'listeA' possédant 0 ou plusieurs 'champB' qui référencent des documents de la '**collectionB**' avec un 'champB' identique

J'ai dans ma base de données **4 collections**:

- **isspositions** : chaque document correspond aux informations sur **la position de l'ISS d'un quiz à une date donnée**, donc les documents sont identifiés de façon unique par leur date.

Exemple, avec la position de l'ISS pour le quiz du 21 mai 2026 :

```
{
  "_id": "6a0f06a63ddc3c06bcd19fc9",
  "date": "2026-05-21T13:20:24.000Z",
  "timestamp": 1779369624,
  "latitude": "24.0326",
  "longitude": "13.2174"
}
```

A noter que les appels au serveur afin de récupérer la position courante de l'ISS, depuis la page d'accueil, ne stockent rien dans la BDD. Seules les positions liées aux quiz y sont.

- **quizzes** : chaque document correspond aux **informations d'un quiz pour une date donnée**, identifié de façon unique par son id et par sa date. Le quiz possède aussi un array de références aux réponses des utilisateurs au quiz (userResponses).

Exemple, avec le quiz du 21 mai 2026 :

```
{
```

```

    "_id": "6a0f06a63ddc3c06bcd19fc8",
    "idQuiz": 18,
    "date": "2026-05-21T13:20:24.000Z",
    "questions": [
      {
        "numQuestion": 0,
        "question": "What historical empire controlled the Fezzan region, including Murzuk, before Italian colonization?",
        "options": [
          "Roman Empire",
          "Ottoman Empire",
          "Persian Empire",
          "British Empire"
        ],
        "indexResponse": 1 // l'indice de l'option correcte à la question
      },
      ...
      {
        "numQuestion": 9,
        "question": "While Murzuk is an oasis, the wider region of southern Libya is known for deposits of what valuable natural resource?",
        "options": [
          "Diamonds",
          "Coal",
          "Oil and natural gas",
          "Copper"
        ],
        "indexResponse": 2
      }
    ],
    "country": "Libya",
    "countryCode": "LY",
    "region": "Murzuq",
    "ocean": false,
    "userResponses": [11, 13]
  }

```

- **users** : chaque document correspond aux **informations d'un utilisateur**. Celui-ci possède un array de références vers ses quiz épinglés (pinnedQuizzes), ainsi que vers ses réponses (userResponses).

Exemple avec l'utilisateur d'id 1 :

```

{
  "_id": "69f2985fc8842a7a717b338a",
  "idUser": 1,
  "username": "a",
  "email": "a@gmail.com",
  "password":
"$2a$10$hVWYw93vwRAQx3RyMVXil.qn0dkRLDR0B9Ne1zwiVYNWaBBhYXuoW",
  "subscribeDate": "2026-04-01T21:41:54.000Z",
  "pinnedQuizzes": [8, 12, 18],
  "userResponses": [1, 5, 10, 13]
}

```

- **userresponses** : chaque document correspond aux **réponses d'un utilisateur à un quiz**.

Exemple, avec les réponses du user d'id 1 au quiz du 21 mai 2026 :

```
{
  "_id": "6a0f2f6ec38b2f506725672d",
  "idUserResponses": 13,
  "idQuiz": 18,
  "idUser": 1,
  "questionsResponses": [
    {
      "numQuestion": 0, // l'indice de la question dans le tableau "questions" du quiz référencé
      "numResponse": 0, // l'indice de l'option répondue par l'utilisateur à la question
      "correct": false
    },
    ...
    {
      "numQuestion": 9,
      "numResponse": 0,
      "correct": false
    }
  ],
  "responseDate": "2026-05-21T16:14:38.425Z",
  "note": 3,
  "comment": "A"
}
```

En terme de **mises à jour** concernant ces données :

- Pour **isspositions**, comme dit précédemment une nouvelle position est insérée à chaque génération de quiz, soit chaque jour. Cette position étant récupérée via l'API externe de Open Notify.
- Pour **quizzes**, on a de même une nouvelle insertion par jour, après génération par l'API externe Gemini.
- Pour **users**, à chaque nouveau profil créé, un user est créé en base de données. Lorsque l'utilisateur modifie ses informations personnelles, la modification se reflète aussi dans la base de données, et lorsqu'il supprime son compte, on va alors d'abord effectuer la suppression des références à ses **userresponses** au sein des array "userResponses" des **quizzes** auxquels il a répondu, puis on va supprimer ses **userresponses**, et enfin on va supprimer le **user** en lui même.
- Pour **userresponses**, celles-ci sont créées lorsqu'un utilisateur poste ses réponses (il n'est alors pas possible de les modifier par la suite), et supprimées sur la suppression de l'utilisateur ayant répondu, comme décrit auparavant.

Description du serveur (Go)

Mon serveur a été implémenté en **Go** et suit une **approche ressources**.

On y retrouve donc les mêmes ressources que celles décrites dans ma base de données.

En terme de composition de handlers selon les répertoires de mon backend :

authentification

- **LoginHandler.go** : prend en charge la route **/api/authentification/login** afin de gérer la connexion d'un utilisateur : il vérifie l'exactitude du login et du mot de passe fourni, et renvoie à l'utilisateur un **cookie contenant un JWT l'identifiant**.
- **LogoutHandler.go** : prend en charge la route **/api/authentification/logout** afin de supprimer le cookie d'un utilisateur (son cookie passe à une date d'expiration immédiate).
- **MeHandler.go** : prend en charge la route **/api/authentification/me**. Lors d'un login ou d'une création de profil, on ne renvoie pas directement en clair le nouvel identifiant. Afin de connaître son statut de connexion, l'application côté client refait alors une requête sur cette route, qui va essentiellement lire le contenu du JWT dans le cookie, et renvoyer l'id ainsi que le nom d'utilisateur de l'utilisateur couramment connecté.
- **SignupHandler.go** : prend en charge la route **/api/authentification/signup**. Le serveur vérifie qu'aucun utilisateur ne possède déjà le même nom et/ou email, insère l'utilisateur en BDD, et renvoie un cookie contenant un JWT comme pour le login.

external_apis

- **CurrentISSHandler.go** : prend en charge la route **/api/external/iss**, appelé de manière périodique par le client et impliquant un appel du serveur à Open Notify, récupérant les coordonnées courantes de l'ISS et les renvoyant directement au client, sans insertion en BDD.
- **GenerateQuizHandler.go** : prend en charge la route **/api/external/generate-quiz**, appelé de manière journalière par une **tâche CRON définie par la GitHub Actions** du répertoire. L'appel est **programmé tous les jours à minuit UTC**, mais dans les faits GitHub applique un **délai de quelques heures**, ce qui fait que la requête et le quiz ne sont générés de manière effective que vers 5/6h du matin, heure française. Comme vu précédemment, le pipeline de création de quiz fait appel à mes 3 APIs externes, et insère en BDD le quiz et sa position correspondante de l'ISS.

resources

Pour **ISSHandler.go**

- **ISSHandler** : prend en charge la route **/api/resources/iss/**, avec dans les faits des requêtes de la forme **/api/resources/iss/date** permettant d'accéder en lecture seule à la position de l'ISS pour une date donnée.

Pour **QuizHandlers.go**

- **QuizHandler** : prend en charge les routes **/api/resources/quizzes**, et **/api/resources/quizzes/**, effectuant une redirection vers un sous-handler selon les cas, parmi :
 - **quizHandlerWithoutParam** pour **/api/resources/quizzes**. Permet de récupérer l'ensemble des quiz, on peut aussi passer à l'URL un paramètre **'?stats=true'** pour obtenir les stats des quiz.
 - **quizHandlerWithId** pour **/api/resources/quizzes/id**. Permet de récupérer le quiz d'id donné, on peut aussi passer à l'URL un paramètre **'?stats=true'** pour obtenir les stats du quiz.
 - **quizHandlerWithDate** pour **/api/resources/quizzes/date**. Permet de récupérer le quiz de la date donnée, on peut aussi passer à l'URL un paramètre **'?stats=true'** pour obtenir les stats du quiz.
- **QuizCommentsHandler** : prend en charge la route **/api/resources/quizzes/comments/**, avec dans les faits des requêtes de la forme **/api/resources/quizzes/comments/id** récupérant les commentaires des réponses au quiz d'id donné.

Pour **UsersHandlers.go**

- **UsersHandler** : prend en charge la route **/api/resources/users/**, effectuant une redirection vers le sous-handler :
 - **userHandlerWithId** pour **/api/resources/users/id**. Permet de manipuler les informations d'un utilisateur donné. Sans le token JWT associé, il n'est possible que de récupérer les infos publiques d'un utilisateur. Avec le token JWT, il est possible de récupérer les infos publiques, de modifier les infos personnelles ou encore de supprimer l'utilisateur.
- **UsersPinQuizHandler** : prend en charge la route **/api/resources/users/pin**. Permet à un utilisateur ayant un JWT d'ajouter un quiz à sa liste de quiz épinglés.
- **UsersUnpinQuizHandler** : prend en charge la route **/api/resources/users/unpin**. Permet à un utilisateur ayant un JWT de retirer un quiz de sa liste de quiz épinglés.
- **UsersRespondedQuizzesHandler** : prend en charge la route **/api/resources/users/responded/**, avec dans les faits des requêtes de la forme **/api/resources/users/responded/id**. Permet de récupérer la liste des identifiants des quiz auxquels l'utilisateur d'id donné a répondu.
- **UsersLeaderboardHandler** : prend en charge la route **/api/resources/users/leaderboard**. Permet de récupérer un classement des utilisateurs calculé à partir de leurs scores totaux, triés par score décroissant puis par ordre alphabétique en cas d'égalité.

Pour **UserResponsesHandlers.go**

- **UserResponsesHandler** : prend en charge les routes **/api/resources/user-responses** et **/api/resources/user-responses/**, effectuant une redirection vers un sous-handler selon les cas, parmi :
 - **userResponsesHandlerWithoutId** pour **/api/resources/user-responses**. Permet de créer une nouvelle réponse utilisateur à un quiz. Lors de la création, les références correspondantes sont également ajoutées à l'utilisateur et au quiz concernés.

- **userResponsesHandlerWithId** pour **/api/resources/user-responses/id**. Permet de récupérer une réponse utilisateur donnée à partir de son identifiant.
- **userResponsesHandlerWithQuizAndUser** pour des requêtes de la forme **/api/resources/user-responses?idquiz='id1'&iduser='id2'**. Permet de récupérer la réponse d'un utilisateur donné à un quiz donné, si elle existe.

À noter que j'ai implémenté l'ensemble des **opérations CRUD sur les ressources**, au début des fichiers contenant les handlers sur la ressource correspondante, mais je n'ai pas implémenté de routes non utilisées et potentiellement dangereuses si accessibles : par exemple je n'ai pas de route pour permettre de supprimer un quiz.

Description des requêtes

Cette section est à relier avec la description précédente des handlers du serveur.
Pour les routes utilisées, je vais donner à chaque fois un exemple de requête et de réponse.

- /api/authentication/login

Requête : **POST**

```
{
  "username": "a",
  "password": "aa"
}
```

Réponse : HTTP 200

```
{
  "message": "login successful"
}
```

+ Cookie de 10min contenant JWT avec id et username de l'utilisateur

- /api/authentication/logout

Requête : **POST**

+ Cookie contenant JWT

Réponse : HTTP 200

```
{
  "message": "logout successful"
}
```

+ Cookie écrasant l'ancien, avec fin immédiate

- /api/authentication/me

Requête : **GET**

+ Cookie contenant JWT

Réponse : HTTP 200

```
{
  "id": 1,
  "username": "a"
}
```

- /api/authentication/signup

Requête : **POST**

```
{
  "username": "a",
  "email": "a@gmail.com",
  "password": "aa"
}
```

Réponse : HTTP 201

```
{
  "message": "signup successful"
}
```

+ Cookie de 10min contenant JWT avec id et username de l'utilisateur

- **api/external/iss**

Requête : GET	Réponse : HTTP 200 <pre>{ "date": "2026-03-20T15:29:18.000+00:00", "timestamp": 1789936153 "latitude": "36.8601" "longitude": "68.7956" }</pre>
----------------------	--

- **/api/external/generate-quiz**

Requête : POST	Réponse : HTTP 201 <pre>{ "message": "new quiz fetched from Gemini and inserted into database" }</pre>
-----------------------	---

- **/api/resources/iss/2026-05-16T12:39:13.000+00:00**

Requête : GET	Réponse : HTTP 200 <pre>{ "date": "2026-05-16T12:39:13.000+00:00", "timestamp": 1778935153 "latitude": "33.8601" "longitude": "64.7956" }</pre>
----------------------	--

- **/api/resources/quizzes**

Requête : GET	Réponse : HTTP 200 Array contenant la liste des quiz, chacun de la même forme que pour /api/resources/quizzes/'id' après
----------------------	--

- **/api/resources/quizzes?stats=true**

Requête : GET	Réponse : HTTP 200 Array contenant la liste des quiz avec leurs stats, chacun de la même forme que pour /api/resources/quizzes/'id'?stats=true après
----------------------	--

- **/api/resources/quizzes/5 et /api/resources/quizzes/2026-04-18T12:58:59Z**

Requête : GET	Réponse : HTTP 200 <pre>{ "idQuiz": 5, "date": "2026-04-18T12:58:59Z", "questions": [{ "numQuestion": 0, "question": "What is the second-largest ocean on Earth by area?", "options": ["Pacific Ocean",</pre>
----------------------	--

	<pre> "Atlantic Ocean", "Indian Ocean", "Arctic Ocean"], "indexResponse": 1 }, ...], "country": "Atlantic Ocean", "countryCode": "Atlantic Ocean", "region": "", "ocean": true, "userResponses": [] }</pre>
--	---

- **/api/resources/quizzes/5?stats=true** et
/api/resources/quizzes/2026-04-18T12:58:59Z?stats=true

Requête : GET	Réponse : HTTP 200 pareil que sans stats, mais avec en plus : <pre> { ... "avgScore": -1, // pas de réponses au quiz, -1 "avgUserNote": -1, // pas de notes au quiz, -1 "nbComments": 0 }</pre>
----------------------	---

- **/api/resources/quizzes/comments/18**

Requête : GET	Réponse : HTTP 200 <pre> [{ "idUser": 1, "username": "a", "date": "2026-05-21T16:14:38.425Z", "note": 3, "comment": "A" }, { "idUser": 5, "username": "TOTHE TOP", "date": "2026-05-21T13:34:47.616Z", "note": 5, "comment": "YAY" }]</pre>
----------------------	--

- /api/resources/users/1

Requête : GET sans cookie + JWT	Réponse : HTTP 200 { "idUser": 1, "username": "a", "subscribeDate": "2026-04-01T21:41:54Z", "userResponses": [1, 5, 10, 13], "pinnedQuizzes": [8, 12, 18] }
Requête : GET avec cookie + JWT	Réponse : HTTP 200 { "idUser": 1, "username": "a", "email": "a@gmail.com", "subscribeDate": "2026-04-01T21:41:54Z", "userResponses": [1, 5, 10, 13], "pinnedQuizzes": [8, 12, 18] }
Requête : PATCH avec cookie + JWT (pour une update groupée) { "username": "a2", "email": "a2@gmail.com", "password": "a2" }	Réponse : HTTP 200 { "message": "user information(s) successfully updated in DB" }
Requête : PATCH avec cookie + JWT (pour une update individuelle) { "username": "a2", }	Réponse : HTTP 200 { "message": "user information(s) successfully updated in DB" }

Requête : DELETE avec cookie + JWT	Réponse : HTTP 200 { "message": "user successfully deleted from DB", }
- /api/resources/users/pin	
Requête : POST { "idQuiz" : 5 }	Réponse : HTTP 200 { "message": "quiz 5 successfully pinned" }
- /api/resources/users/unpin	
Requête : POST { "idQuiz" : 5 }	Réponse : HTTP 200 { "message": "quiz 5 successfully unpinned" }
- /api/resources/users/responded/1	
Requête : GET	Réponse : HTTP 200 [8, 12, 9, 18]
- /api/resources/users/leaderboard	
Requête : GET	Réponse : HTTP 200 [{ "idUser": 5, "username": "TOTHE TOP", "totalScore": 24, "nbQuizResponded": 4 }, { "idUser": 3, "username": "c", "totalScore": 6, "nbQuizResponded": 1 }, { "idUser": 1, "username": "a", "totalScore": 5, "nbQuizResponded": 4 }, { "idUser": 2, "username": "b", "totalScore": 4, "nbQuizResponded": 3 }, { "idUser": 4, "username": "d", "totalScore": 0, "nbQuizResponded": 0 }]

```
}  
]
```

- /api/resources/user-responses

Requête : **POST**

```
{  
  "comment" : "", // pas de commentaire  
  "idQuiz" : 18,  
  "idUser" : 2,  
  "note" : 0, // pas de note donnée au quiz  
  "questionsResponses" : [  
    {numQuestion: 0, numResponse: 0, correct: false},  
    {numQuestion: 1, numResponse: 0, correct: false},  
    {numQuestion: 2, numResponse: 0, correct: false},  
    {numQuestion: 3, numResponse: 0, correct: false},  
    {numQuestion: 4, numResponse: 0, correct: false},  
    {numQuestion: 5, numResponse: 0, correct: false},  
    {numQuestion: 6, numResponse: 0, correct: false},  
    {numQuestion: 7, numResponse: 0, correct: false},  
    {numQuestion: 8, numResponse: 0, correct: false},  
    {numQuestion: 9, numResponse: 0, correct: false}  
  ]  
}
```

Réponse : HTTP 201

```
{  
  "message" : "user responses successfully created in  
DB",  
  "idUserResponses": 14  
}
```

- /api/resources/user-responses/14

Requête : **GET**

Réponse : HTTP 200

```
{  
  "idUserResponses": 14,  
  "idQuiz": 18,  
  "idUser": 2,  
  "questionsResponses": [  
    {"numQuestion": 0, "numResponse": 0, "correct": false},  
    {"numQuestion": 1, "numResponse": 0, "correct": false},  
    {"numQuestion": 2, "numResponse": 0, "correct": false},  
    {"numQuestion": 3, "numResponse": 0, "correct": false},  
    {"numQuestion": 4, "numResponse": 0, "correct": false},  
    {"numQuestion": 5, "numResponse": 0, "correct": false},  
    {"numQuestion": 6, "numResponse": 0, "correct": false},  
    {"numQuestion": 7, "numResponse": 0, "correct": false},  
    {"numQuestion": 8, "numResponse": 0, "correct": false},  
    {"numQuestion": 9, "numResponse": 0, "correct": false}  
  ],  
  "responseDate": "2026-05-23T09:40:34.948Z",  
  "note": 0,  
  "comment": ""  
}
```

- /api/resources/user-responses?idquiz=18&iduser=2

Requête : **GET**

Réponse : HTTP 200

renvoie la même réponse que avant, pour
/api/resources/user-responses/14

Description du client (React)

Voici les routes :

- / pour la page d'accueil.
- /quizzes pour la page de l'ensemble des quizzes.
- /quiz/:id pour la page d'un quiz spécifique, dont l'id est donné.
- /leaderboard pour la page du classement des utilisateurs du site.
- /profile/:id pour la page de profil d'un utilisateur dont l'id est donné.
- /login pour la page de connexion.
- /signup pour la page de création de profil.

Je vais pour chaque écran spécifier le contenu et les appels au serveur faits par les composants affichés (les composants mineurs ne sont pas décrits).

/ (Home)

Après le paragraphe de présentation du site, on a un composant **ISSPosition** correspondant à une carte interactive, offrant à l'utilisateur la possibilité d'afficher la position courante de l'ISS, ou d'afficher la position passée de l'ISS au moment de la création du quiz du jour. La position courante est obtenue toutes les 5 secondes par polling, en faisant un **GET** `/api/external/iss`, tandis que la position du quiz est obtenue seulement au premier affichage avec un **GET** `/api/resources/iss/'date_du_jour'`.

Ensuite est affiché le composant **Quiz**, récupérant éventuellement en cache : le quiz et les réponses couramment entrées de l'utilisateur. Si le quiz n'est pas présent dans le cache, un appel **GET** `/api/resources/quizzes/'parameter'` est effectué, avec parameter = la date courante si sur la page d'accueil (ici le cas), ou l'id du quiz si sur la page spécifique. Si l'utilisateur est connecté, 2 appels supplémentaires sont faits : **GET** `/api/resources/user-responses?idquiz='idQuiz'&iduser='connectedId'`, avec l'id du quiz concerné et l'id de l'utilisateur connecté, pour obtenir les éventuelles réponses de l'utilisateur au quiz, et **GET** `/api/resources/users/'connectedId'` pour récupérer les quiz épinglés. Une **ERREUR 404** est alors retournée **si l'utilisateur connecté n'a pas encore répondu au quiz, c'est normal**, je n'ai pas décidé de faire taire cette erreur dans la console.

- **Si l'utilisateur n'a pas encore répondu** : le sous-composant **ToDoQuiz**, est affiché, permettant de donner ses réponses au quiz, et à la validation un popup **CompletedQuizPopup** apparaît, permettant de voir son score, mettre un commentaire et une note au quiz, et de poster ses réponses, faisant alors un **POST** `/api/resources/user-responses` (délégué à **ToDoQuiz**). À chaque modification d'une réponse, du commentaire, ou de la note avant d'avoir posté, les **réponses sont mises en cache**.
- **Si l'utilisateur a déjà répondu au quiz**, le sous-composant **RespondedQuiz** est affiché, montrant alors ses réponses, son score, et son éventuel commentaire et/ou note au quiz.

Dans tous les cas, les commentaires du quiz sont affichés par le sous-composant **QuizComments**, faisant un **GET** `/api/resources/quizzes/comments/'idQuiz'` pour les récupérer.

/quizzes (Quizzes)

Le composant **Quizzes** récupère l'ensemble des quiz du site, d'abord via le cache, puis via un **GET** `/api/resources/quizzes?stats=true` si non présent en cache. Si l'utilisateur est connecté, il récupère aussi la liste des ids des quiz répondus avec **GET** `/api/resources/users/responded/'connectedId'`, et la liste des quiz épinglés, avec **GET** `/api/resources/users/'connectedId'`.

Alors 2 possibilités selon ce que l'utilisateur choisit :

- Pour le sous-composant **QuizzesList**, affiché par défaut, celui-ci permet de voir la liste complète des quiz du site. Pour chaque quiz sont alors affichés comme informations la date du quiz, le pays et la région concernés, le nombre de réponses au quiz, le score moyen des réponses, et la note moyenne donnée par les utilisateurs à ce quiz. En outre, si l'utilisateur est connecté, il peut alors également épingler/désépingler un quiz affiché dans la liste, faisant un **POST** `/api/resources/users/pin` ou **POST** `/api/resources/users/unpin`. L'utilisateur a également la possibilité de filtrer ces quiz affichés, que ce soit textuellement en entrant un pays ou une région, ou en sélectionnant une date précise. À cela peut se combiner le fait d'afficher les quiz selon un ordre sur une des colonnes, par exemple par défaut les quiz sont affichés selon un ordre décroissant sur la date des quiz (du plus récent au plus ancien).
- Pour le sous-composant **QuizzesMap**, celui-ci récupère alors les positions de chaque quiz, d'abord en essayant via le cache, puis par appels à **GET** `/api/resources/iss/'date_quiz'` si pas en cache. Chaque quiz est alors placé sur la carte selon la position de l'ISS au moment du quiz.

/quiz/:id (QuizPage)

Un composant **QuizPage** est affiché, lui-même contenant un **Quiz**. **Le reste est identique au quiz sur la page d'accueil.**

/leaderboard (LeaderboardPage)

Le composant **Leaderboard** est affiché dans cette page, récupérant directement le classement trié par le serveur via **GET** `/api/resources/users/leaderboard`, et affichant le classement des utilisateurs.

/profile/:id (ProfilePage)

Le composant **Profile** va d'abord récupérer les infos du profile avec **GET** `/api/resources/users/'id'` (rappel : si l'utilisateur est connecté, cet appel renvoie toutes ses infos, sinon seulement une partie publique de ses infos), puis on récupère ensuite chacune des réponses de l'utilisateur dont le profil est regardé, avec **GET** `/api/resources/user-responses/'idUserR'`.

2 cas sont alors possibles :

- **Le profil regardé n'est pas celui de l'utilisateur couramment connecté** : le sous-composant **PublicProfile** est alors affiché avec son nom d'utilisateur, sa date de création de profil, ses statistiques avec son nombre de quiz effectués, son score moyen et ses notes moyennes données, en plus de son activité récente : ses quiz

effectués du plus ancien au plus récent, avec ses éventuels commentaires et/ou notes.

- **Le profil regardé est celui de l'utilisateur couramment connecté** : le sous-composant **PrivateProfile** est alors affiché, avec en plus des informations publiques, l'adresse e-mail et la liste de quiz épinglés. L'utilisateur peut alors :
 - Désépingler un quiz, via un **POST** `/api/resources/users/unpin`.
 - Passer sur la vue de modification de ses informations personnelles et sauvegarder un changement, via un **PATCH** `/api/resources/users/'id'`.
 - Se déconnecter, via un **POST** `/api/authentication/logout`.
 - Supprimer son profil, via un **DELETE** `/api/resources/users/'id'`.

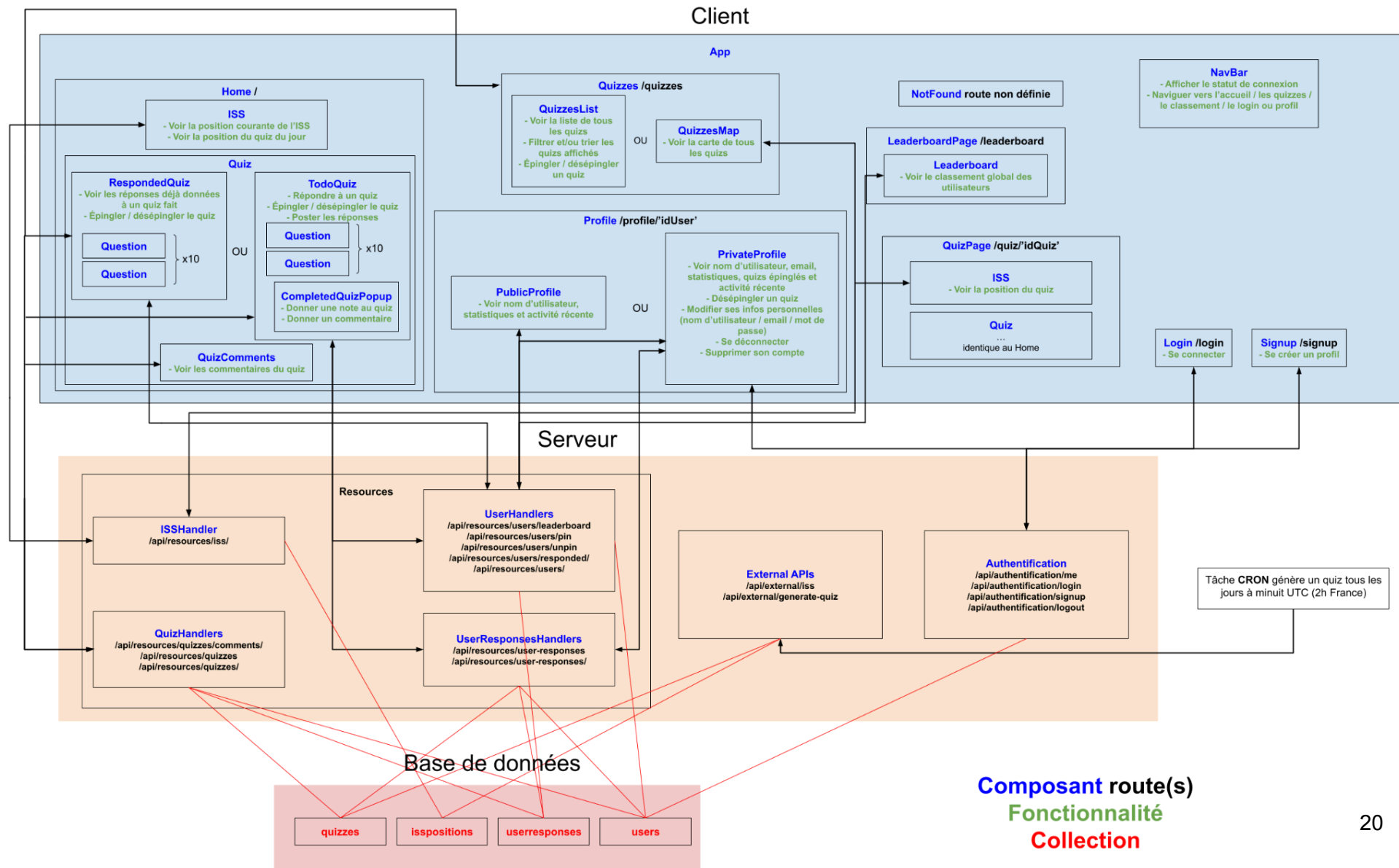
/login (Login)

Permet de se connecter et d'accéder à son profil, via **POST** `/api/authentication/login`, puis **GET** `/api/authentication/me` pour récupérer l'id et le username de l'utilisateur nouvellement connecté.

/signup (Signup)

Permet de se créer un compte et d'accéder à son profil, via **POST** `/api/authentication/signup`, puis **GET** `/api/authentication/me` pour récupérer l'id et le username de l'utilisateur nouvellement connecté.

Schéma général de l'application



Composant route(s)
Fonctionnalité
Collection

Utilisation d'IA

J'avais deux approches différentes vis-à-vis de mon utilisation d'IA durant le projet :

- **Côté backend j'ai essentiellement tout fait à la main**, mis à part quelques appels à des APIs avec lesquelles je n'étais pas familier, typiquement pour la génération de quiz avec Gemini, au début du projet pour l'interaction avec la base de données MongoDB, ou encore pour avoir un peu d'aide concernant la gestion des cookies et de JWT (quand bien même pour précisément ça, j'ai vraiment essayé d'aller regarder par moi même les docs / des tutos pour comprendre comment les utiliser).
- **Côté frontend, je me suis surtout concentré sur la gestion des états, des appels au serveur, des éventuels traitements sur les données récupérées** (sur ça j'ai aussi eu besoin d'un peu d'aide lorsque j'étais bloqué), et de la **structure générale du layout**. J'ai délégué une grosse partie de la création du CSS à Claude et ChatGPT (désolé le CSS n'est pas ma passion), et une partie du HTML. Par exemple en termes d'HTML, j'ai fait une grosse partie de la page d'accueil et des quiz individuels, tandis que j'ai plus délégué pour le HTML des profils, du leaderboard et en partie de la liste générale des quiz. Malgré ça, j'ai quand même essayé de comprendre ce qu'ils me faisaient et j'ai quasiment toujours ajouté / modifié ce qu'il me renvoyaient car le résultat ne correspondait pas forcément à ce que je voulais, ce qui fait que je devais quand même souvent intervenir.